

I numeri finiti e il calcolo numerico

Calcolo Numerico
a.a. 2025-26

Elena Loli Piccolomini

5 settembre 2025

Outline

Accuratezza di un risultato numerico

Errori nel calcolo numerico

Rappresentazione dei numeri

Conversione della rappresentazione di un numero reale

I numeri finiti

Aritmetica floating point

Bibliografia

- ▶ Gladwell, Nagy, Ferguson, Introduction to scientific computing using Matlab cap. 2
- ▶ M. Heat, Scientific Computing: an introductory survey, cap.1

Errori nel calcolo numerico

Nella risoluzione di un problema numerico al calcolatore si possono avere diversi tipi di errore:

- ▶ **Errore di misura**, dovuto alle imperfezioni dello strumento di misura dei dati del problema.
- ▶ **Errore di troncamento**, quando un procedimento infinito viene realizzato come procedimento finito. (esempio: calcolo del valore di una funzione tramite sviluppo in serie).
- ▶ **Errore algoritmico**, dovuto al propagarsi degli errori di arrotondamento sulle singole operazioni in un procedimento complesso.
- ▶ **Errore inerente**, dovuto al fatto che i dati di un problema non sempre appartengono all'insieme $\mathbb{F}$ dei numeri rappresentabili e quindi vengono approssimati.

Accuratezza di un risultato numerico

Misura dell'accuratezza:

- ▶ **errore assoluto** $E_a = | \text{ris. approssimato} - \text{ris. esatto} |$
- ▶ **errore relativo** $E_r = \frac{\text{errore assoluto}}{|\text{risultato esatto}|}$ (ris. esatto $\neq 0$)
- ▶ **errore percentuale** $E_p = (E_r \times 100)\%$

Accuratezza di un risultato numerico (cont.)

Sono molto importanti nella decisione se accettare o meno la soluzione di un problema numerico.

Fuori dal contesto, il valore dell'errore assoluto ha poco significato.

$\alpha = 0.3 \cdot 10^1$	$\alpha^* = 0.31 \cdot 10^1$	$E_a = 0.1$	$E_r = 0.3333.. \cdot 10^{-1}$ $E_p = 3.33..%$
$\alpha = 0.3 \cdot 10^{-3}$	$\alpha^* = 0.31 \cdot 10^{-3}$	$E_a = 0.1 \cdot 10^{-4}$	$E_r = 0.3333.. \cdot 10^{-1}$ $E_p = 3.33..%$
$\alpha = 0.3 \cdot 10^4$	$\alpha^* = 0.31 \cdot 10^4$	$E_a = 0.1 \cdot 10^3$	$E_r = 0.3333.. \cdot 10^{-1}$ $E_p = 3.33..%$

Rappresentazione in base β

Fissato un numero β intero maggiore di 1, rappresentiamo nella base β un numero reale qualunque α .

► rappresentazione di tipo misto

$$\alpha = \pm(\alpha_0\alpha_1\alpha_2\dots\alpha_p.\alpha_{p+1}\alpha_{p+2}\dots)_\beta = \sum_{k=0}^p \alpha_k \beta^{p-k} + \sum_{k=1}^{\infty} \alpha_{k+p} \beta^{-k} \quad (1)$$

► rappresentazione normalizzata

$$\alpha = \pm(. \alpha_1 \alpha_2 \dots) \beta^p$$

Altre basi

- ▶ Base 8, cifre 0, 1, 2, 3, 4, 5, 6, 7

$$(0.36207)_8 = 3 \times 8^{-1} + 6 \times 8^{-2} + \dots = (0.47286\dots)_{10}$$

- ▶ Base 2, cifre 0, 1, o sui calcolatori “off” e “on” (“bit” = binary digit)

$$(0.111)_2 = 1 \times 2^{-1} + 1 \times 2^{-2} + 1 \times 2^{-3} = (0.875)_{10}$$

Conversione in base 2: numeri interi

Conversione del numero naturale da base 10 a base 2 (sistema binario con il **metodo delle divisioni successive** per la conversione)

Esempio

$$(34)_{10} = (100010)_2 = 0 \times 2^0 + 1 \times 2^1 + 0 \times 2^2 + 0 \times 2^3 + 0 \times 2^4 + 1 \times 2^5$$

$$34:2=17 \text{ resto } 0$$

$$17:2=8 \text{ resto } 1$$

$$8:2=4 \text{ resto } 0$$

$$4:2=2 \text{ resto } 0$$

$$2:2=1 \text{ resto } 0$$

$$1:2=0 \text{ resto } 1$$



- ▶ Si ottengono le cifre di rappresentazione di α in base β dalla meno significativa alla più significativa.
- ▶ Pertanto, scrivendo i resti delle divisioni nell'ordine inverso a quello in cui sono stati ottenuti si ottiene la rappresentazione in forma sintetica di α in base β .

Conversione in base 2: numeri reali < 1

- Si utilizza il **metodo delle moltiplicazioni successive**

base 2 $0.1 \times 2 = 0.2$ p. intera 0

$0.2 \times 2 = 0.4$ p. intera 0

$0.4 \times 2 = 0.8$ p. intera 0

$0.8 \times 2 = 1.6$ p. intera 1

$0.6 \times 2 = 1.2$ p. intera 1

$0.2 \times 2 = 0.4$ p. intera 0

...

$$(0.1)_{10} = (0.000\overline{1100})_2$$

- Ci si arresta o perchè la parte frazionaria diventa nulla o perchè si è raggiunto un numero di cifre sufficienti.
- non è detto che se un numero ha rappresentazione finita in base 10 altrettanto accade in base β

Conversione in base 2: Numeri reali

Esempio: $\alpha = -25.357$

- ▶ Si converte la parte intera $25 = (11001)_2$
- ▶ Si converte la parte frazionaria $.357 = (.011)_2$
- ▶ $\alpha == (-11001.011)_2$

Rappresentazione in memoria: i numeri interi

- ▶ Con N bit sono rappresentabili i numeri naturali da 0 a 2^{N-1} .
- ▶ **Rappresentazione complemento alla base**: un numero negativo $-x$ viene rappresentato in base b in $t + 1$ posizioni come $b^{t+1} - x$

Esempi:

- ▶ $x = -31152$ è rappresentato come $100000000 - 31152 = 99968848$
- ▶ $x = -10000000$ è rappresentato come 90000000

Rappresentazione dei I numeri reali

- **Rappresentazione scientifica normalizzata di un numero reale:** ogni numero reale $x \in \mathbb{R}$ può essere rappresentato come

$$x = \pm(0.d_1d_2d_3\dots)\beta^p = \pm \sum_{i=1}^{\infty} (d_i\beta^{-i})\beta^p \quad (2)$$

dove

- p è un numero intero
- le cifre d_i verificano le condizioni:
 - i) $0 \leq d_i \leq \beta - 1$
 - ii) $d_1 \neq 0$ e le d_i non sono tutte uguali a $\beta - 1$ a partire da un indice in poi.

I numeri reali (cont.)

- ▶ La rappresentazione di x nella forma $\pm(0.d_1d_2d_3\dots)\beta^p$ è chiamata **rappresentazione normalizzata**
- ▶ Il numero $m = 0.d_1d_2d_3\dots$ viene detto comunemente la **mantissa** di x
- ▶ β^p la **parte esponente**
- ▶ Il numero p è detto anche **caratteristica** (o esponente) di x
- ▶ β è detto **base**
- ▶ Le d_i sono dette **cifre della rappresentazione**

Rappresentazione in memoria dei numeri reali: Sistema floating point

- ▶ A causa della sua capacità finita, un calcolatore non è in grado di rappresentare tutto l'insieme dei numeri reali. Si pone pertanto il problema di definire, per ogni x rappresentato nella forma 2, una sua rappresentazione approssimata nel calcolatore.
- ▶ Si tratta di un fatto tecnico, ma con importanti implicazioni nel calcolo numerico.
- ▶ Un metodo divenuto ormai usuale per il calcolo scientifico e noto come **sistema floating-point** o virgola mobile.
- ▶ Esso permette la rappresentazione di un ampio intervallo della retta reale con una distribuzione uniforme degli errori relativi. L'ampiezza effettiva dell'intervallo dipende dal particolare calcolatore su cui la procedura è implementata.

Sistema floating point (cont.)

Si definisce **insieme dei numeri macchina (floating-point)** con t cifre significative, base β e range (L, U) , l'insieme dei numeri reali definito nel modo seguente

$$\mathbb{F}(\beta, t, L, U) = \{0\} \cup \left\{ x \in \mathbb{R} = \text{sign}(x)\beta^p \sum_{i=1}^t d_i \beta^{-i} \right\}$$

ove t, β sono interi positivi con $\beta \geq 2$. Si ha inoltre

$$\begin{aligned} 0 \leq d_i \leq \beta - 1, \quad i = 1, 2, \dots \\ d_1 \neq 0, \quad L \leq p \leq U \end{aligned}$$

Usualmente U è positivo e L negativo.

Sistema floating point (cont.)

- ▶ In rappresentazione posizionale un numero macchina $x \neq 0$ viene denotato con $x = \pm.d_1d_2 \dots d_t\beta^p$
- ▶ La maggior parte dei calcolatori ha la possibilità di operare con lunghezze diverse di t , a cui corrispondono, ad esempio, la semplice e la doppia precisione.
- ▶ E' importante osservare che l'insieme $\mathbb{F}$ non è un insieme continuo e neppure infinito.

Sistema floating point (cont.)

I numeri dell'insieme $\mathbb{F}$ sono ugualmente spazati tra le successive potenze di β , ma non su tutto l'intervallo.

Esempio $\beta = 2$, $t = 3$, $L = -1$, $U = 2$.

$$\mathbb{F} = \{0\} \cup \{0.100 \times 2^p, 0.101 \times 2^p, 0.110 \times 2^p, 0.111 \times 2^p, p = -1, 0, 1, 2\}$$

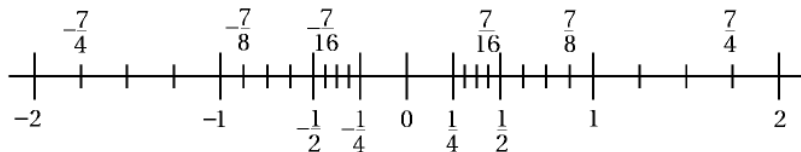
dove 0.100, 0.101, 0.110, 0.111 sono tutte le possibili mantisse e p il valore dell'esponente.

Sistema floating point (cont.)

Sono 33 numeri compreso lo zero

$.100(-1)$	$.101(-1)$	$.110(-1)$	$.111(-1)$
$1/4$	$5/16$	$6/16$	$7/16$
$.100(0)$	$.101(0)$	$.110(0)$	$.111(0)$
$1/2$	$5/8$	$6/8$	$7/8$
$.100(1)$	$.101(1)$	$.110(1)$	$.111(1)$
1	$5/4$	$6/4$	$7/4$
$.100(2)$	$.101(2)$	$.110(2)$	$.111(2)$
2	$5/2$	$6/2$	$7/2$

Sistema floating point (cont.)



Risulta evidente l'aumento dell'ampiezza degli intervalli definiti dal valore p , in ognuno dei quali vengono localizzati i 4 valori della mantissa; ne risulta di conseguenza una diradazione delle suddivisioni verso gli estremi sinistro e destro della retta reale.

Sistema floating point (cont.)

Le grandezze fondamentali che definiscono l'insieme dei numeri finiti, oltre alla base di rappresentazione β , sono:

- ▶ L e U che definiscono l'ordine di grandezza dei numeri rappresentabili e in genere $L \sim U$;
- ▶ il numero t di cifre della mantissa che fissa la precisione di rappresentazione di ogni numero; infatti la retta reale viene suddivisa in intervalli $[\beta^p; \beta^{p+1}]$ di ampiezza crescente esponenzialmente con il valore p , ed in ognuno di questi intervalli vengono rappresentati lo stesso numero $(\beta - 1)\beta^{t-1}$ di valori. Si ha quindi una suddivisione molto densa per valori vicini allo 0 e più rada per valori grandi in valore assoluto.

Sistema floating point (cont.)

- ▶ Tutti i numeri reali all'interno di un sotto-intervallo tra due valori rappresentabili vengono approssimati da uno dei due estremi, quindi maggiore è il numero t di cifre della mantissa, minore sarà l'ampiezza dei sotto-intervalli e quindi migliore l'approssimazione introdotta. La normalizzazione di numeri floating point causa un "salto" intorno allo 0 nell'intervallo dei reali.
- ▶ La più piccola variazione della mantissa è $u = \beta^{-t}$ (ULP=unit of last position).
- ▶ La precisione del numero floating point dipende dalla lunghezza della mantissa

Sistema floating point (cont.)

- ▶ Gli n bit (o posizioni) disponibili per la memorizzazione di un numero finito vengono suddivisi tra le t cifre della mantissa e l'esponente p che può assumere $U - L + 1$ configurazioni diverse, più un bit per il segno del numero.
- ▶ Nel caso della rappresentazione binaria, sarà sempre $d_1 = 1$, per cui può essere sottointeso senza mai essere fisicamente rappresentato.

Sistema floating point (cont.)

- ▶ Si noti che la normalizzazione della mantissa permette di sfruttare al meglio le cifre disponibili, evitando di rappresentare esplicitamente gli 0 non significativi. Ad esempio 0.000124 viene rappresentato come 0.124×10^{-3} .
- ▶ La virgola quindi non ha una posizione fissa tra la parte intera e quella non, ma varia; per questo motivo tale rappresentazione è detta **floating-point** o a virgola mobile.

Sistema floating point (cont.)

Come rappresentare un numero reale positivo x in un sistema di numeri macchina $\mathbb{F}(\beta, t, L, U)$?

- ▶ Il numero x è tale che $L \leq p \leq U$ e $d_i = 0$ per $i > t$; allora x è un numero macchina ed è rappresentato esattamente.
- ▶ $p \notin [L, U]$; il numero non può essere rappresentato esattamente. Se $p < L$, si dice che si verifica un underflow; solitamente si assume come valore approssimato del numero x il numero zero. Se $p > U$ si verifica un overflow e solitamente non si effettua nessuna approssimazione, ma il sistema di calcolo dà un avvertimento più drastico, come ad esempio, l'arresto del calcolo.

Sistema floating point (cont.)

- La caratteristica $p \in [L, U]$, ma le cifre d_i , per $i > t$, non sono tutte nulle. In questo caso si pone il problema di scegliere un suo rappresentante in $\mathbb{F}$. Tale operazione viene indicata comunemente come operazione di arrotondamento (rounding) o troncamento (truncation).

Se il numero è infinito, al numero viene associato il suo float ottenuto troncando (o arrotondando) la mantissa e viene rappresentato in memoria il float ottenuto:

$$x = 1.333333 \dots = 0.133333 \dots \cdot 10^1 = fl(x) = 0.133333 \cdot 10^1$$

Errori di rappresentazione

- ▶ Errore assoluto di arrotondamento

$$|fl(x) - x| < \beta^{p-t}$$

- ▶ Errore relativo di arrotondamento

$$\frac{|fl(x) - x|}{|x|} < \frac{1}{2}\beta^{1-t}$$

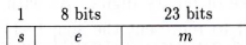
La quantità $eps = \frac{1}{2}\beta^{1-t}$ è detta **precisione macchina** nel fissato sistema floating point. La sua importanza numerica è data dalla seguente caratterizzazione: eps è il più piccolo numero macchina positivo tale che

$$fl(1 + eps) > 1$$

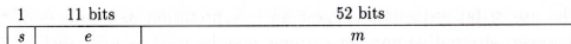
Standard IEEE

- ▶ Importanza di definire uno standard della rappresentazione.
- ▶ IEEE computer society (Institute of Electrical and Electronic Engineers) crea “IEEE standard for binary floating Arithmetic” (spesso riferito come IEEE 754) nel 1985.
- ▶ Rappresentazione dei formati IEEE standard in memoria

Single Precision Format: format width $N = 32$ bits



Double Precision Format: format width $N = 64$ bits



Standard IEEE (cont.)

- ▶ Formato singola precisione $\mathbb{F}_{(2,24,-128,127)}$, 32 bit.
- ▶ Formato doppia precisione $\mathbb{F}_{(2,53,-1024,1023)}$, 64 bit.
- ▶ Nella codifica IEEE standard esistono anche sequenze di bit per rappresentare i valori $+0$, -0 , $+\infty$, $-\infty$, e quantità simboliche chiamate NaN (Not a Number). Queste quantità sono utilizzate per gestire “speciali” operazioni, tipo la radice quadrata di un numero negativo, altrimenti non rappresentabili

Aritmetica floating point

- ▶ Le operazioni eseguite sul calcolatore (calcolo numerico) possono essere eseguite solo con numeri rappresentabili sul calcolatore stesso.
- ▶ Poichè $\mathbb{F}$ è un sottoinsieme di $\mathbb{R}$, le usuali operazioni aritmetiche sono definite anche per operandi in $\mathbb{F}$, ma il loro risultato, in generale, non sta in $\mathbb{F}$.
- ▶ Il risultato esatto è arrotondato ad un numero che stia in $\mathbb{F}$.

Aritmetica floating point (cont.)

- ▶ Operazione aritmetica reale $\cdot : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$
- ▶ operazione floating-point (o di macchina): $\odot : \mathbb{F} \times \mathbb{F} \rightarrow \mathbb{F}$

$$x \odot y = fl(x \cdot y)$$

- ▶ Ogni operazione provoca in generale un errore, detto **errore di arrotondamento**, molto piccolo

$$\left| \frac{(x \odot y) - (x \cdot y)}{x \cdot y} \right| < eps$$

Aritmetica floating point (cont.)

Realizzazione hardware di un'operazione floating point (con registro con precisione estesa) $x \oplus y$

- ▶ Eseguo l'operazione esatta $z = x + y$
- ▶ Effettuo il troncamento del risultato $x \oplus y = fl(z)$

Non valgono le proprietà commutativa e associativa nell'aritmetica floating point.

- ▶ Sulla maggior parte dei computer le operazioni aritmetiche (passo 1) sono fatte utilizzando più bit di quelli utilizzati per memorizzare il risultato.
- ▶ Il formato intermedio esteso non è accessibile all'utente, ma viene memorizzato in un registro interno alla CPU.
- ▶ Una volta che il calcolo è stato fatto in precisione estesa, il risultato viene arrotondato alla precisione del risultato (passo 2)

Aritmetica floating point

Il risultato di un'operazione in aritmetica floating point può essere differente rispetto al risultato della stessa operazione in aritmetica esatta.

- ▶ Addizione o sottrazione: lo spostamento della mantissa può causare perdita di cifre o di accuratezza.
- ▶ Moltiplicazione: Il prodotto di due numeri con t -cifre di mantissa ha al più $2t$ cifre, quindi il risultato può non essere rappresentabile.
- ▶ Divisione: Il quoziente di due numeri con t cifre di mantissa può contenere più di t cifre come per esempio la rappresentazione binaria di $1/10$

Example: $\beta = 10$, $t = 6$

$x = 192.403$, $y = 0.635782$, $fl(x) = 0.192403 \cdot 10^3$, $fl(y) = 0.635782 \cdot 10^0$

- ▶ $z = fl(x) + fl(y) = (0.192403 + 0.000635782) \cdot 10^3 = 0.193038782 \cdot 10^3$
 $fl(z) = 0.193039 \cdot 10^3$ The last two digits of y do not affect the result, and with even smaller exponent, y could have had no effect on the result.
- ▶ $w = fl(x) * fl(y) = (0.635782 \cdot 0.192403) \cdot 10^3 = 0.122326364146 \cdot 10^3$
 $fl(w) = 0.122326 \cdot 10^3$ che elimina la metà delle cifre..

Aritmetica floating point

Il risultato in aritmetica reale puo'non essere rappresentabile an ch
perche' l'esponente p e' esterno al suo range di rappresentazione:

- ▶ $p > U$ Overflow
- ▶ $p < L$ Underflow

Aritmetica floating point (cont.)

Esempio: $x = 2.15 \times 10^{12}, y = 1.25 \times 10^{-5}$
 $z = x - y$

Viene calcolata come:

$$x = 2.15 \times 10^{12}$$

$$y = 0.00000000000000000125 \times 10^{12}$$

$$x - y = 2.149999999999999999 \times 10^{12}$$

arrotondato a 2.15×10^{12}

Aritmetica floating point (cont.)



Esempio di **errore catastrofico**

```
function approx
% valuta:  e = exp(1) = lim_{n->infinity} (1 + 1/n)^n
e = exp(1);
fprintf('\n      n          f(n)          error\n');
for n=logspace(0,16,9)
    f = (1+1/n)^n;
    fprintf('%9.1e  %14.10f  %14.10f\n',n,f,abs(f-e));
end
```

Calcolo del numero:
$$e = \lim_{n \rightarrow \infty} \left(1 + \frac{1}{n}\right)^n$$

Aritmetica floating point (cont.)



>> epprox

n	f(n)	<u>error</u>
1.0e+000	2.00000000000	0.7182818285
1.0e+002	2.7048138294	0.0134679990
1.0e+004	2.7181459268	0.0001359016
1.0e+006	2.7182804691	0.0000013594
1.0e+008	2.7182817983	0.0000000301
1.0e+010	2.7182820532	0.0000002248
1.0e+012	2.7185234960	0.0002416676
1.0e+014	2.7161100341	0.0021717944
1.0e+016	1.00000000000	1.7182818285

Aritmetica floating point (cont.)

Errori di arrotondamento:

1. Calcolo di $1/n$: errore di arrotondamento. Non provoca un errore grande rispetto al valore di $1/n$.
2. Calcolo di $1 + 1/n$: provoca errore di arrotondamento grande quando n è grande. L'errore assoluto è piccolo rispetto a 1, ma grande rispetto a $1/n$. Per $n > 1016$, $1 + 1/n = 1$ in aritmetica doppia precisione.
3. $(1 + 1/n)^n$: amplifica l'errore commesso al punto 2

Il problema in questo caso è quello di sommare due numeri di grandezza molto diversa.

Aritmetica floating point (cont.)

$$c = a + b, a \gg b$$

$$c = a - b, a \ll b (a < 0)$$

Provocano grandi errori di arrotondamento

Esempio: $c = a + b, a = x.xxxx... \times 10^0, b = y.yyy... \times 10^{-8}$

La somma nell'aritmetica floating point viene fatta utilizzando una precisione estesa per la mantissa e riportando i due numeri allo stesso esponente (**il maggiore**):

$$\begin{array}{r} \text{available precision} \\ x.xxx \text{ xxxx xxxx xxxx} \\ + \quad 0.000 \text{ 0000 yyyy yyyy yyyy} \\ \hline = \quad x.xxx \text{ xxxx zzzz zzzz } \underbrace{\text{yyyy yyyy}}_{\text{lost digits}} \end{array}$$

Le cifre più significative di a si mantengono, quelle di b no.